



Divide y Vencerás

Estructura de Datos y Algoritmos I

Profesora: Elba Karen Sáenz García

Divide y Vencerás

Divide y vencerás es una técnica de diseño de algoritmos que consiste en resolver un problema a partir de la solución de subproblemas del mismo tipo, pero de menor tamaño.



Consta de tres pasos:

Dividir el problema en subproblemas del mismo tipo

Resolver cada subproblema de forma recursiva. Hasta llegar al caso base

Combinar las soluciones obtenidas para la solución al problema original



Paso 1 Dividir

- Plantear el problema de forma que pueda descomponerse en k *subproblemas del mismo tipo, pero de menor tamaño.*

Si el tamaño de la entrada es n , *debemos dividir el problema en k subproblemas*

(donde $1 < k \leq n$) *cada uno con una entrada cuyo tamaño aproximado es de n/k .*

A esta tarea se le conoce como división.



Paso 2 - Vencer o conquistar

- Resolver independientemente todos los subproblemas, ya sea directamente si son elementales o bien de forma recursiva.
- Que el tamaño de los subproblemas sea estrictamente menor que el tamaño original del problema nos garantiza la convergencia hacia los casos elementales, también denominados *casos base*, en los que se utiliza un método directo para obtener la solución



Paso 3 Combinar

3-Combinar las soluciones obtenidas en el paso anterior para construir la solución del problema original.

- Utilizar parámetros de acumulación para poder hacer esta combinación de soluciones.

DV (x)

```
if (x es suficientemente pequeño) {  
    return algoritmo_específico(x);  
} else {  
    descomponer x en  $\{x_1, \dots, x_k\}$   
    for i = 1 to k  
         $y_i \leftarrow \text{DV}(x_i)$ 
```

Ejemplos

- Búsqueda Binaria
- Ordenación por Mezcla (Merge Sort)
- Ordenación Rápida (Quick Sort)
- Multiplicación de números enteros lagos
- Multiplicación matrices etc..
- Búsqueda del máximo

Ejemplos en clase

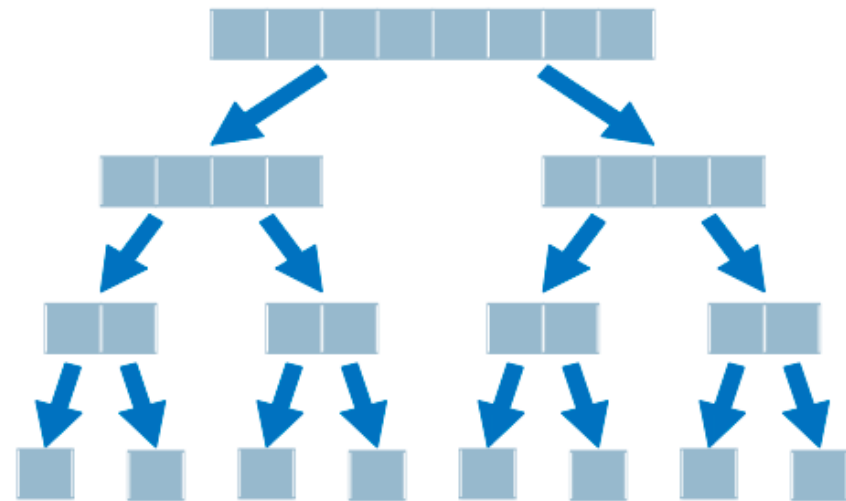
Divide y vencerás – Encontrar máximo

A:

-3	24	11	-13	...	33	-7	12	1
----	----	----	-----	-----	----	----	----	---

1. Divide el array en dos partes
2. Encontrar el máximo en cada parte
3. Comparar ambos números y devolver el mayor.

Es aplicado mediante llamadas recursivas



$$\text{Max}(44, 30) = 44$$

4	17	44	10	30	25	6
---	----	----	----	----	----	---

$$\text{Max}(17, 44) = 44$$

4	17	44
---	----	----

$$\text{Max}(30, 25) = 30$$

10	30	25	6
----	----	----	---

$$\text{Max}(4, 17) = 17$$

4	17
---	----

44

$$\text{Max}(10, 30) = 30$$

10	30
----	----

$$\text{Max}(25, 6) = 25$$

25	6
----	---

4	17
---	----

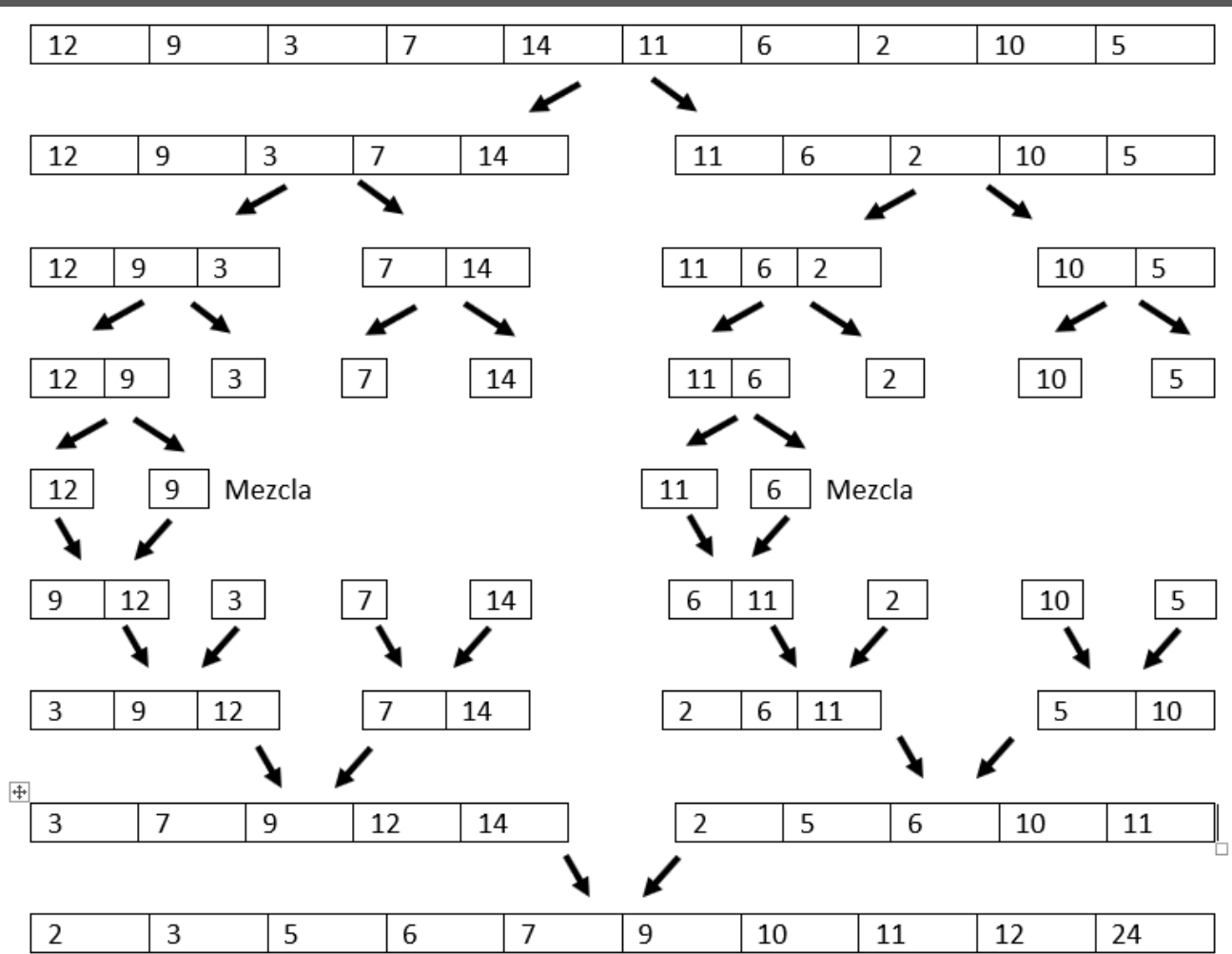
10	30	25	6
----	----	----	---

```
encuentraMax (vector, i, j)
  Si (i==j)
    retorna vector[i]
  Sino
    jmed = (i + j) / 2
    max_left = encuentraMax (vector, i, jmed)
    max_right = encuentraMax(vector, jmed+1, j)
  Fin Sino
  Si (max_left > max_right)
    retorna max_left
  Si no
    retorna max_right;
  Fin Sino
FIN
```

Ordenación por Mezcla Merge Sort

EDAI

Elba Karen Sáenz García



Merge Sort

Las tres fases en este algoritmo son:

- **Divide:** Se divide una secuencia A de n elementos a ordenar en dos sub-secuencias de $n/2$ elementos.
- Para dividirlo en 2 se encuentra un número q .

El cálculo se hace $q = \left\lfloor \frac{p+r}{2} \right\rfloor$.

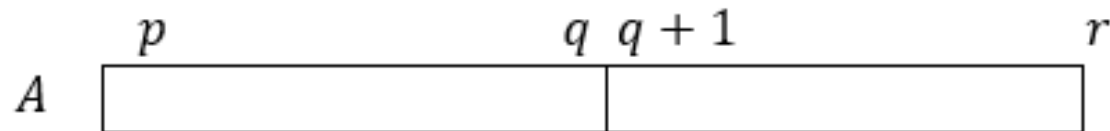


Figura 1.1.

Merge Sort

- **Conquista:** Ordena las dos sub-secuencias de forma recursiva utilizando el algoritmo **MergeSort()**.

El caso base o término de la recursión ocurre cuando la secuencia a ser ordenada tiene solo un elemento y éste se supone ordenado.

- **Combina:** Mezcla las dos sub-secuencias ordenadas para obtener la solución.

Algoritmo

MergeSort(A,p,r)

Inicio

Si $p < r$ entonces } // Divide

$$q = \left\lfloor \frac{p+r}{2} \right\rfloor$$

MergeSort(A, p , q) } //Conquista

MergeSort(A, q+1 , r) }

Merge(A,p,q,r) // Mezcla o combina

Fin Si

Fin

Merge(A,p,q,r)

Inicio

Formar sub-arreglo Izq[0,..., q-p] y sub-arreglo Der[0,...,r-q]

Copiar contenido de A[p...q] a Izq[0,..., q-p] y A[q+1...r] a Der[0,...,r-q -1]

i=0

j=0

Para k=p hasta r

Si (j >= r-q) ó (i < q-p+1 y Izq[i] < Der[j]) entonces

A[k]=Izq[k]

i=i+1

En otro caso

A[k]=Der[j]

j=j+1

Fin Si

Fin Para

Fin

Algoritmo Merge-sort

- Sea A un arreglo de n elementos y p, r índices del rango a ordenar.

```
Merge-Sort(A, p, r)          --> T(n)
if ( p < r ) {                -->  $\Theta(1)$ 
    q = parteEntera((p+r)/2);  -->  $\Theta(1)$ 
    Merge-Sort(A, p, q);       --> T(n/2)
    Merge-Sort(A, q+1,r);      --> T(n/2)
    Merge(A, p, q, r);         -->  $\Theta(n)$ 
}
```

¿Cuál es el costo de este algoritmo?

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n/2) + T(n/2) + n & \text{if } n > 1 \end{cases}$$

$$T(n) = T(n/2) + T(n/2) + n$$

$$O(n \log n)$$



Algoritmo de Karatsuba

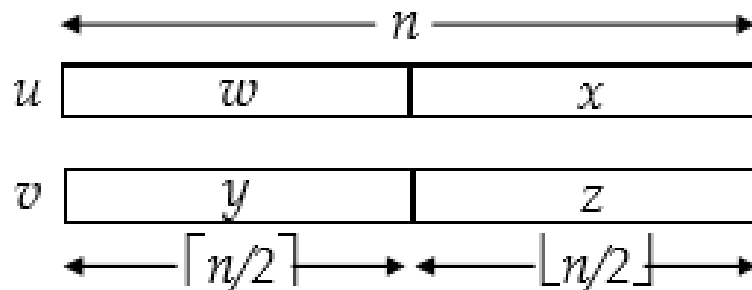
- Técnica de divide y vencerás para la multiplicación de enteros muy grandes.
- Sean u y v dos enteros de n cifras.
- Se descomponen en mitades de igual tamaño:

Multiplicación de enteros grandes

- $1234 * 5678 = (12 * 100 + 34) * (56 * 100 + 78) = (12 * 10^2 + 34) * (56 * 10^2 + 78) =$
 $u * v = 12 * 56 * 10000 + (12 * 78 + 34 * 56) * 100 + 34 * 78$

$$\left. \begin{array}{l} u = 10^s w + x \\ v = 10^s y + z \end{array} \right\} \text{ con } 0 \leq x < 10^s, \quad 0 \leq z < 10^s, \quad y$$

$$s = \lfloor n/2 \rfloor$$



$$uv = 10^{2s} wy + 10^s (wz + xy) + xz$$

Análisis

- Si n se supone par $t(n) = 4t(n/2) + \Theta(n)$

Las sumas, multiplicaciones por potencias de 10 y divisiones por potencias de 10: tiempo lineal.

Operación módulo una potencia de 10: tiempo lineal (puede hacerse con una división, una multiplicación y una resta).

La solución de esta recurrencia es:

$$t(n) \in O(n^2)$$

Multiplicación de Matrices

$$\begin{array}{|c|c|} \hline A_{11} & A_{12} \\ \hline A_{21} & A_{22} \\ \hline \end{array} \times \begin{array}{|c|c|} \hline B_{11} & B_{12} \\ \hline B_{21} & B_{22} \\ \hline \end{array} = \begin{array}{|c|c|} \hline C_{11} & C_{12} \\ \hline C_{21} & C_{22} \\ \hline \end{array}$$

$$\begin{aligned}
 C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\
 C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\
 C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\
 C_{22} &= A_{21}B_{12} + A_{22}B_{22}
 \end{aligned}$$

Aplicamos **divide y vencerás**:

Cada matriz de **$n \times n$** es dividida en cuatro submatrices de tamaño **$(n/2) \times (n/2)$** : A_{ij} , B_{ij} y C_{ij} .

Es necesario resolver 8 problemas de tamaño **$n/2$** .

n es potencia de dos

mulMat(A, B)

$A[0][0] * B[0][0]$ si $n=1$

$C11 = \text{mulMat}(A11, B11) + \text{mulMat}(A12, B21)$ si $n > 1$

$C12 = \text{mulMat}(A11, B12) + \text{mulMat}(A12, B22)$

$C21 = \text{mulMat}(A21, B11) + \text{mulMat}(A22, B21)$

$C22 = \text{mulMat}(A21, B12) + \text{mulMat}(A22, B22)$

Análisis

La combinación de los resultados requiere un $O(n^2)$.

$$t(n) = 8 \cdot t(n/2) + a \cdot n^2$$

Resolviéndolo obtenemos que $t(n)$ es $O(n^3)$.